Lab 07: One Small Stepper Motor, One Giant State Machine

ECE 484: Embedded Systems Design

Grade	Score	Signature
Attendance: James Nidda		
Attendance: Gabe Litteken		
Attendance:		
Demonstration		

Overview

In this lab, you will design an embedded system that will lock and unlock a door. Unlocking and locking the door will be simulated by rotating a stepper motor 180 degrees CCW and CW, respectively. Changing the state of the lock will require the use of three push buttons to enter an activate lock and deactive the lock. For demonstration and debugging purposes, you will need to send the current state of the lock and the input code on the LCD screen.

Deliverables

1. System Demonstration (10 points)
2. Completed Worksheet submitted to Canvas (5 points)
3. Commented Source Code submitted to Canvas (5 points)

Instructions

Your embedded system will consist of the following hardware components:

1. An LCD screen connected to a I2C/TWI module. (Remember to use pull-up resistors for SCL and SDA lines).
2. Three pushbuttons with pull-up circuits. The inputs from these pushbuttons will be connected to three GPIO of your choice.
3. A stepper motor connected to the UNL2003 stepper motor driver module (included in your kit). You will use PINC0:4 to connect to IN1:4 on the stepper motor driver.

Starter code for this lab is provided in the assignment page on Canvas. For the software, you will first configure your GPIO and then configure Timer 1 to operate in CTC mode and trigger an output compare match interrupt every 1ms. By default, the starter code has enabled the TWI module in your microcontroller and intialized the LCD screen. The Timer 1 ISR will read the three pushbutton inputs; pass the raw inputs to the `softwareDebounce()` function; and call the `lockStateMachine()` function if there is a falling edge on any of the three pushbuttons.

Scanned with CamScanner

The majority of the code for this lab will be written in the `lockStateMachine()`. In the `lockStateMachine()` function, you will be responsible for designing a state machine to handle the correct sequence of button presses (i.e. code). The system will default to start in the unlock state. A sequence of button 1, button 2, and button 3 presses will start the lock process. The lock process will use the `stepperMotorMachine()` function to move the stepper motor 180° CCW. You will need to use the `stepperDirection` $\in \{-1, 1\}$ variable to control the direction the stepper motor rotates. To ensure the proper frequency for the stepper motor, you will need to use the *_delay_ms*() function to ensure that the stepper motor increments at a rate of 100Hz. After the lock process has completed, you will need to display "lock" on the first line of the LCD screen. Once your system is locked, a sequence of button 3, button 2, and button 1 presses will start the unlock process. The unlock process will use the `stepperMotorMachine()` function to move the stepper motor 180° CW. Once the unlock process has completed, you will output "unlock" to the LCD screen.

For debugging and demonstration purposes, your state machine will also need to output the current code input to the system on the second line of the LCD screen. The second line of the LCD screen should read CODE:123 or CODE:321 when the code for lock and unlock, respectively, is input to the system. Your state machine should also clear the code if an incorrect button press is detected. **HINT:** write a function that will clear the code digits and call this function in your state machine when an incorrect button press sequence is detected.

Scanned with CamScanner

Demonstration Rubric

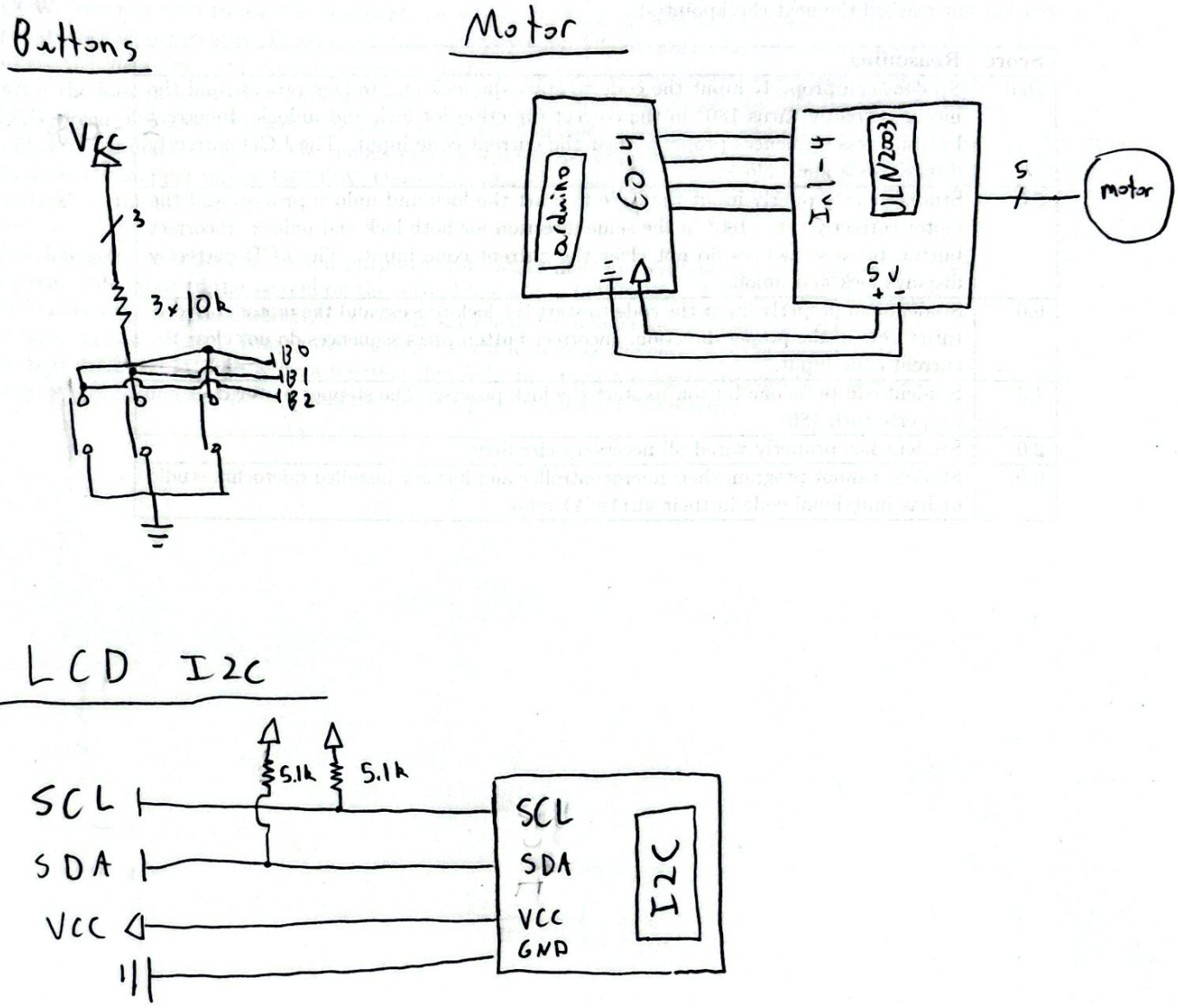
Please demonstrate your system to Dr. Swanson or the TA and they will record your demonstration score at the top of your lab manual. Checkpoint scores are provided in the following rubric. Scores between checkpoints may be awarded if a student has exceeded the previous checkpoint but has not reached the next checkpoint(s).

Score	Reasoning
10.0	Student can properly input the code to start the lock and unlock process and the motor correctly turns 180° in the correct direction for lock and unlock. Incorrect button press sequences properly clear the current code input. The LCD correctly displays lock and unlock.
8.0	Student can properly input the code to start the lock and unlock process and the motor correctly turns 180° in the same direction for both lock and unlock. Incorrect button press sequences do not clear the current code input. The LCD correctly displays lock and unlock.
6.0	Student can properly input the code to start the lock process and the motor correctly turns 180o in the proper direction. Incorrect button press sequences do not clear the current code input.
4.0	Student can press one button to start the lock process. The stepper motor does not properly turn 180°.
2.0	Student has properly wired all necessary circuitry.
0.0	Student cannot program their microcontroller and has not installed microchip studio or has functional code in their `while(1)` loop.

Scanned with CamScanner

Circuit Diagram

Draw a circuit diagram for your system. Make sure to label all major modules, resistance values, and signal lines.



Scanned with CamScanner

Design Questions

1. In your state machine, the `stepperStateMachine()` function uses full-step mode for the UNL2003 driver. If `stepperStateMachine()` were to instead implement half-step mode, how many steps would be necessary to rotate the stepper motor 900°?

$180° \rightarrow 1024$ steps $\quad \frac{900°}{180°} = 5$

$(1024)(5) = 5120$ steps (full steps)

half steps: $\checkmark \cdot 2 = 10{,}240$ steps

2. From an energy conservation perspective, what is one downside of using the stepper motor in your embedded system? (*Hint: Check the stepper motor driver module*)

In order to maintain its position, the stepper motor must constantly use power.

3. What would happen if you turned off the GPIO that control the stepper motor when the stepper motor is not rotating?

Theoretically, without the gear train in the mix, the rotor would be able to rotate freely as there is no electro-magnetic holding the rotor in place.

Scanned with CamScanner